Pointers - Pjesa 1

Programim në gjuhën C

Alion Emini

kurseinformatike.com

Përmbajtja

1. Hyrje
2. Deklarimi i pointer-ave
3. Operatorët & dhe *
   - Operatori i adresës
   - Pointeri NULL
   - Printimi i adresave
   - Operatori i indirektimit
4. Caktimi i Pointerave
5. Pointer-at dhe Funksionet
6. Prioriteti i operatorëve me *
7. Përmbledhje

Përmbajtja

1. Hyrje
2. Deklarimi i pointer-ave
3. Operatorët & dhe *
4. Caktimi i Pointerave
5. Pointer-at dhe Funksionet
6. Prioriteti i operatorëve me *
7. Përmbledhje

Hyrje

Pse pointerat janë të rëndësishëm?

Pointerat janë një nga konceptet më të rëndësishme në C, sepse lidhin programimin me mënyrën se si funksionon memoria.

Ide kryesore

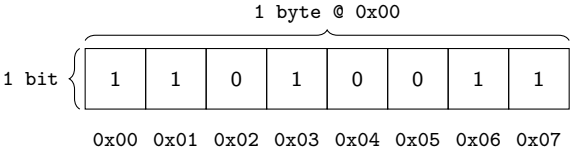
- Pointer = **adresë** në memorie
- Variabël pointer = variabël që ruan një adresë
- Me pointera mund të:
  - modifikojmë variabla përmes funksioneve
  - punojmë me arrays në mënyrë efikase
  - kthejmë adresa nga funksionet

Memoria, byte dhe adresa

Memoria në nivel bazë

Në shumicën e kompjuterëve modernë, memoria kryesore mund të mendohet si një sekuencë **byte**-sh. Çdo byte:

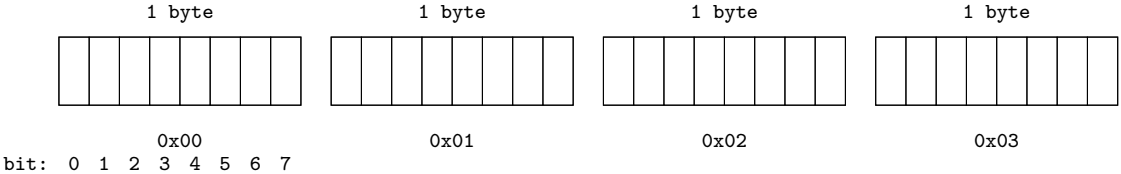
- ruan zakonisht 8 bit informacioni
- ka një **adresë unike**



Memoria si shumë byte (adresimi)

Adresa i përket byte-it (jo bit-it)

Memoria është një varg byte-sh. Çdo **byte** ka adresën e vet; bitët janë vetëm **brenda** atij byte-i (bit 0–7).



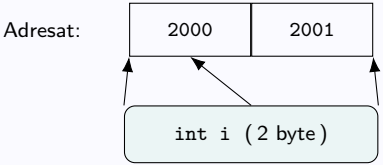
Variablat dhe adresa e tyre

Koncepti

Çdo variabël zë një ose më shumë byte në memorie. Adresa e byte-it të parë quhet **adresa e variablës**.

Shembull

Nëse variabla `i` zë adresat 2000 dhe 2001, atëherë adresa e saj është **2000**.



Adresa e `i` = 2000

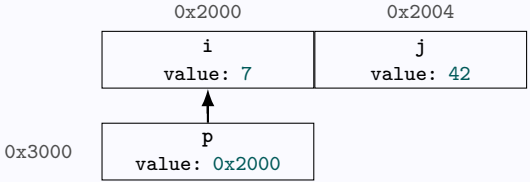
Çfarë është një pointer?

Përkufizim

Një pointer nuk është asgjë më shumë se një **adresë**. Një variabël pointer është një variabël që ruan një adresë të një variabli ose të një pointeri tjetër.

Si e lexojmë

Nëse pointeri `p` ruan adresën e variablës `i`, themi që `p` **punton (points)** te `i`.



Përmbajtja

1 Hyrje

2 Deklarimi i pointer-ave

3 Operatorët & dhe *

4 Caktimi i Pointerave

5 Pointer-at dhe Funksionet

6 Prioriteti i operatorëve me *

7 Përmbledhje

Deklarimi i një variabli pointer

Rregulli

Një pointer deklarohet si një variabël normale, por emri i tij paraprihet nga `*`.

Shembull

```
int *p;
```

- `p` është pointer
- `p` mund të puntojë drejt objekteve të tipit `int`

Terminologji

Përdorim shpesh fjalën **objekt** dhe jo vetëm **variabël**, sepse pointeri mund të tregojë edhe drejt zone memorieje që nuk i përket një variabli të zakonshëm.

Deklarime të përziera

Deklarim në një rresht

Pointerat mund të deklarohen bashkë me variabla të tjera.

Shembull

```
int i, j, a[10], b[20], *p, *q;
```

- i, j → variabla int
- a, b → arrays të int
- p, q → pointera drejt int

Kujdes

Ylli (*) lidhet me **emrin e variablës**, jo me të gjithë rreshtin.

Tipi i referencës së pointerit

Ideja

Çdo pointer në C ka një **tip reference** (çfarë objekti mund të tregojë).

Shembuj

```
int *p; /* tregon vetem drejt int */
double *q; /* tregon vetem drejt double */
char *r; /* tregon vetem drejt char */
```

Vëzhgim

Nuk ka kufizim praktik mbi tipin e referencës: një pointer mund të puntojë edhe drejt një pointeri tjetër.

Përmbajtja

1. Hyrje
2. Deklarimi i pointer-ave
3. Operatorët & dhe *
   - Operatori i adresës
   - Pointeri NULL
   - Printimi i adresave
   - Operatori i indirektimit
4. Caktimi i Pointerave
5. Pointer-at dhe Funksionet

Dy operatorët bazë për pointera

Operatori i adresës: &

Nëse `x` është variabël, atëherë `&x` është adresa e saj në memorie.

Operatori i indirektimit: *

Nëse `p` është pointer, atëherë `*p` është objekti drejt të cilit `p` punton.

Lexim intuitiv

`&x` = "adresa e x"
`*p` = "ajo që ndodhet te adresa e ruajtur në p"

Përmbajtja

1. Hyrje
2. Deklarimi i pointer-ave
3. Operatorët & dhe *
   - Operatori i adresës
   - Pointeri NULL
   - Printimi i adresave
   - Operatori i indirektimit
4. Caktimi i Pointerave
5. Pointer-at dhe Funksionet
6. Prioriteti i operatorëve me *
7. Përmbledhje

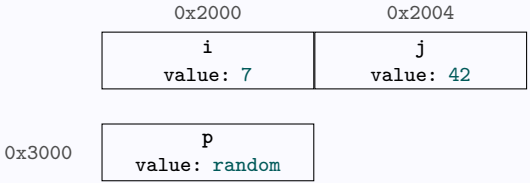
Inicializimi i pointerit me operatorin & I

Parim

Deklarimi i pointerit rezervon vend për pointerin, por **nuk e bën atë të puntojë një variabël**.

Pointer i painicializuar

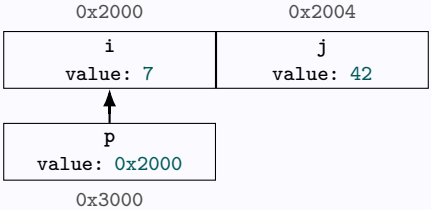
```
int i = 7, j = 42;
int *p; /* ende nuk tregon drejt nje objekti te caktuar */
```



Inicializimi i pointerit me operatorin & II

Inicializim i saktë

```
int i = 7, j = 42;
int *p = &i;
```



Inicializim në momentin e deklarimit

Mënyra të sakta

Mund ta inicializojmë pointerin direkt gjatë deklarimit.

Variant 1

```
int i;
int *p = &i;
```

Variant 2

```
int i, *p = &i;
```

Praktikë e mirë

Inicializo pointerat sa më herët që të jetë e mundur, për të shmangur përdorimin e pointerave të painicializuar.

Përmbajtja

1. Hyrje
2. Deklarimi i pointer-ave
3. Operatorët & dhe *
   - Operatori i adresës
   - Pointeri NULL
   - Printimi i adresave
   - Operatori i indirektimit
4. Caktimi i Pointerave
5. Pointer-at dhe Funksionet
6. Prioriteti i operatorëve me *
7. Përmbledhje

NULL si vlerë e veçantë për pointera I

Çfarë është NULL?

NULL është një vlerë e veçantë që përdoret për të treguar se pointeri **nuk punton te asnjë objekt i vlefshëm**.

Inicializim i sigurt

```
int *p = NULL;
```

NULL si vlerë e veçantë për pointera II

Pse është praktikë e mirë?

Nëse ende nuk kemi një variabël ku duhet të puntojë pointeri, është më mirë ta vendosim në NULL sesa ta lëmë të painicializuar.

Kontroll para dereferencimit

```
if (p != NULL) {
    printf("%d\n", *p);
}
```

NULL $\neq$ pointer i painicializuar

Pointer i painicializuar

```c
int *p; /* gabim: vlera e panjohur */
printf("%d", *p); /* UB */
```

Pointer NULL

```c
int *p = NULL;
printf("%d", *p); /* GABIM (NULL dereference) */
```

Problemi

- p ka vlerë të papërcaktuar
- nuk dimë ku punton
- dereferencimi shkakton sjellje të papërcaktuar

Vëzhgim

- p ka një gjendje të qartë (NULL)
- prapë nuk lejohet dereferencimi
- por është më e lehtë të kontrollohet me if (p != NULL)

Ideja kryesore

Të dyja janë gabim për dereferencim, por NULL është një gjendje e qëllimshme dhe e kontrollueshme, ndërsa pointeri i painicializuar është shumë më i rrezikshëm.

Përmbajtja

1. Hyrje
2. Deklarimi i pointer-ave
3. Operatorët & dhe *
   - Operatori i adresës
   - Pointeri NULL
   - Printimi i adresave
   - Operatori i indirektimit
4. Caktimi i Pointerave
5. Pointer-at dhe Funksionet
6. Prioriteti i operatorëve me *
7. Përmbledhje

Si printohet një adresë në C

Rregulli

Për të printuar një adresë me `printf`, përdorim formatin `%p`.

Shembull i saktë

```c
int i = 7, *p = &i;

printf("Adresa e i = %p\n", (void *)&i);
printf("Vlera e p = %p\n", (void *)p);
```

Çfarë vërejmë

- `(void *)&i` dhe `(void *)p` zakonisht printojnë të njëjtën adresë

Ideja kryesore

`p` ruan një adresë; `&i` prodhon adresën e `i`.

Pse bëjmë cast te `(void *)` me `%p`? I

Arsyeja

Specifikuesi `%p` i `printf` pret argument të tipit `void *`. Prandaj përdorim cast të qartë: `(void *)ptr`.

Praktikë e mirë

```
printf("%p\n", (void *)p);
printf("%p\n", (void *)&i);
```

Pse bëjmë cast te `(void *)` me `%p`? II

Gabime të zakonshme

```
printf("%d\n", p); /* GABIM */
printf("%x\n", p); /* GABIM */
```

Kujdes

- Forma e adresës në ekran (hex, shkronja të vogla/të mëdha, prefiksi `0x`) mund të ndryshojë sipas sistemit/compiler-it.
- Ne e përdorim `%p` vetëm për ta parë adresën në mënyrë të sigurt.

Përmbajtja

1. Hyrje
2. Deklarimi i pointer-ave
3. Operatorët & dhe *
   - Operatori i adresës
   - Pointeri NULL
   - Printimi i adresave
   - Operatori i indirektimit
4. Caktimi i Pointerave
5. Pointer-at dhe Funksionet
6. Prioriteti i operatorëve me *
7. Përmbledhje

Leximi i vlerës përmes *

Ideja

Nëse `p` punton te `i`, atëherë `*p` përfaqëson `i`.

Shembull

```
int i = 7, j = 42;
int *p = &i;
printf("%d\n", *p); // 7
```

Rezultati

`printf` shfaq **vlerën e** `i`, jo adresën e saj.

Marrëdhënia mes & dhe *

```
j = *&i; /* e njejte me j = i; */
```

*p si alias për i I

Parim

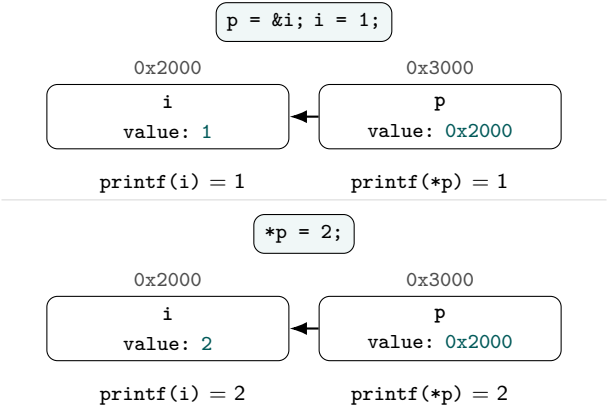
Për sa kohë që p punton te i, shprehja *p është një **alias** për i.

Demonstrim

```
p = &i;
i = 1;

printf("\%d\n", i); /* 1 */
printf("\%d\n", *p); /* 1 */

*p = 2;

printf("\%d\n", i); /* 2 */
printf("\%d\n", *p); /* 2 */
```

*p si alias për i II

Çfarë ndodh në memorie?



Gabim kritik: dereferencim i pointerit të painicializuar I

Mos bëni kurrë këtë

```
int *p;
printf("%d", *p);
```

Shpjegim:

- p nuk ka adresë të vlefshme
- përdorimi i *p shkakton **sjellje të papërcaktuar**
- programi mund të:
  - printojë vlera të çuditshme
  - dështojë
  - sillet në mënyrë të paparashikueshme

Gabim kritik: dereferencim i pointerit të painicializuar II

Edhe më e rrezikshme

```
int *p;
*p = 1;
```

Shpjegim:

- p është i painicializuar, prandaj nuk tregon në një adresë të vlefshme në memorie.
- programi mund të crash-ojë
- mund të korruptojë memorie (të prishë të dhëna diku tjetër)
- mund të sillet në mënyrë të paparashikueshme

Përmbajtja

1 Hyrje

2 Deklarimi i pointer-ave

3 Operatorët & dhe *

4 Caktimi i Pointerave

5 Pointer-at dhe Funksionet

6 Prioriteti i operatorëve me *

7 Përmbledhje

Kopjimi i pointerave (q = p) I

Rregulli

C lejon kopjimin e pointerave me të njëjtin tip.

Shembull

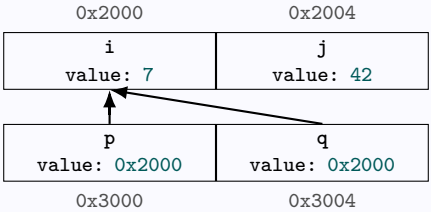
```
int i = 7, j = 42, *p, *q;

p = &i;
q = p;
```

Rezultati

Pas q = p, të dy pointerat tregojnë te i njëjti objekt.

Kopjimi i pointerave (q = p) II

Diagram



Kopjimi i pointerave (q = p) III

Pasojë

`*p = 1;` dhe `*q = 2;` të dyja ndryshojnë `i`.

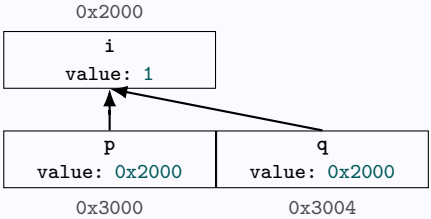
Shembull

```
*p = 1; /* i behet 1 */
*q = 2; /* i behet 2 */
```

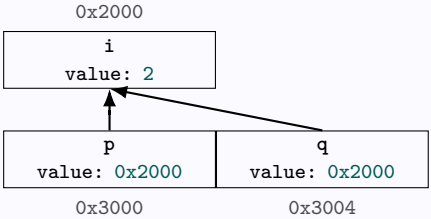
Kopjimi i pointerave (q = p) IV

Diagrami

Pas `*p = 1;`



Pas `*q = 2;`



q = p ≠ *q = *p I

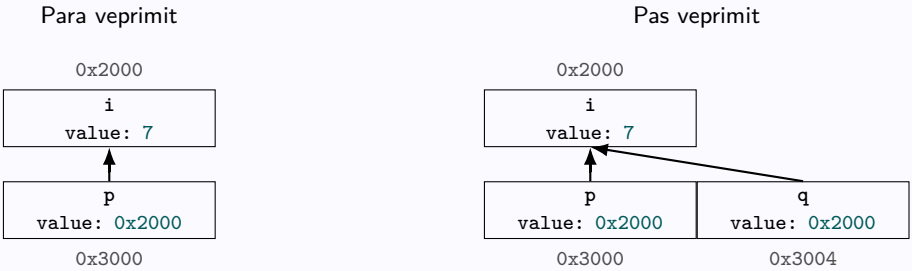
1: Caktim pointerash

```
int i = 7, *p = &i;
int *q = p;
```

q kopjon **vlerën** e pointerit p (pra adresën ku p punton).

q = p ≠ *q = *p ||

Diagram: para/pas int *q = p



q = p ≠ *q = *p III

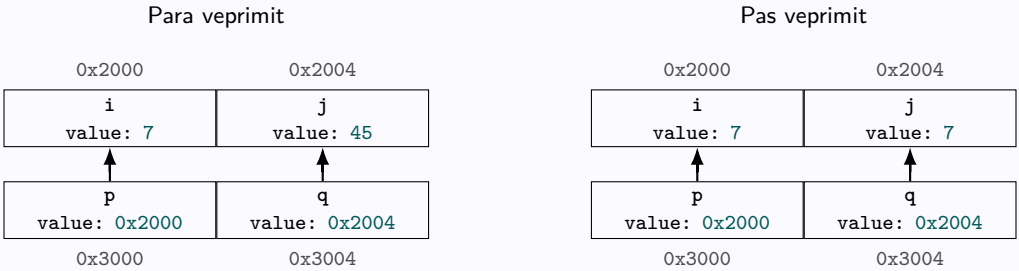
2: Caktim vlerash

```
1
2 int i = 7, j = 45, *p = &i, *q = &j;
3 *q = *p;
```

`*q = *p` kopjon **vlerën** e objektit që `p` punton te objekti që `q` punton.

q = p ≠ *q = *p IV

Diagram: para/pas *q = *p



q = p ≠ *q = *p V

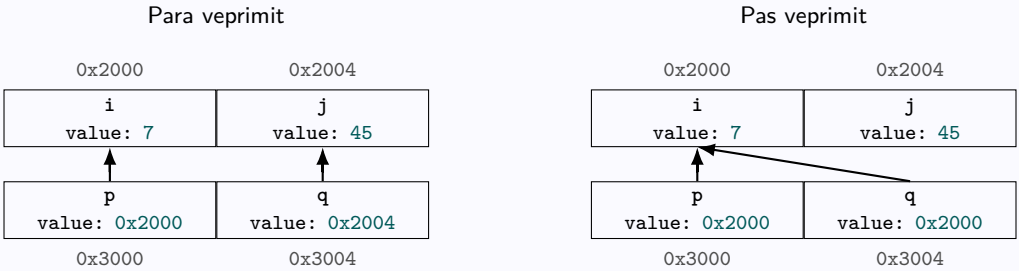
3: Pointeri ndryshon (jo vlera)

```
int i = 7, j = 45, *p = &i, *q = &j;
q = p;
```

Këtu ndryshon q (tani punton te i); j mbetet i pandryshuar.

q = p ≠ *q = *p VI

Diagram: para/pas q = p



Përmbajtja

1 Hyrje

2 Deklarimi i pointer-ave

3 Operatorët & dhe *

4 Caktimi i Pointerave

5 Pointer-at dhe Funksionet

6 Prioriteti i operatorëve me *

7 Përmbledhje

Pse kalojmë pointera te funksionet?

Problemi

C i kalon argumentet **sipas vlerës**, prandaj një funksion normalisht nuk mund të ndryshojë variablat e dërguara si argumente.

Zgjidhja

Në vend të vlerës së variablës `x`, i kalojmë funksionit `&x` (adresën e saj).

Si funksionon

- Parametri deklarohet si pointer
- Brenda funksionit, `*p` bëhet alias për `x`
- Funksioni mund ta lexojë dhe ta ndryshojë `x`

Shembull: decompose

Funksioni

```c
void decompose(double x, long *int_part, double *frac_part) {
    *int_part = (long) x;
    *frac_part = x - *int_part;
}
```

Ideja

Funksioni ndan numrin `x` në:

- pjesën e plotë (`*int_part`)
- pjesën fraksionare (`*frac_part`)

Prototipi

```c
void decompose(double x, long *int_part, double *frac_part);
```

Thirrja e decompose

Thirrja

```
decompose(3.14159, &i, &d);
```

Çfarë kalon te funksioni

- `x` merr kopjen e `3.14159`
- `int_part` merr adresën e `i`
- `frac_part` merr adresën e `d`

decompose: gjendja pas ekzekutimit

Brenda funksionit

```
*int_part = (long) x; /* i = 3 */
*frac_part = x - *int_part;/* d = 0.14159 */
```

Pas kthimit

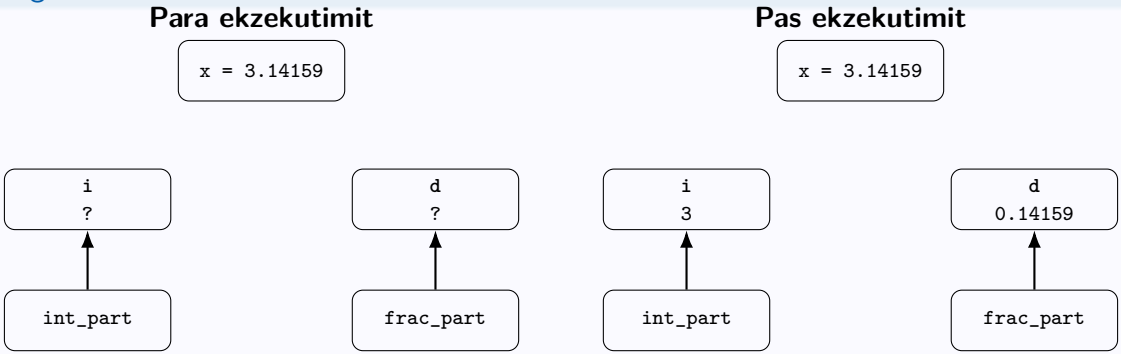
- `i = 3`
- `d = 0.14159`

Thelbi

Funksioni modifikon `i` dhe `d` **indirekt**, përmes pointerave.

decompose: para/pas ekzekutimit të trupit të funksionit

Diagram



scanf dhe pointerat I

Ideja

scanf është shembulli klasik i funksionit që kërkon pointera.

Përdorimi i saktë

```
int i;
scanf("%d", &i);
```

Pse duhet &i?

Sepse scanf duhet të dijë **ku** ta vendosë vlerën e lexuar.

scanf dhe pointerat II

Kur kemi pointer

```
int i, *p;
p = &i;
scanf("%d", p); /* sakte */
```

Gabim

```
scanf("%d", &p); /*** GABIM ***/
```

Kjo do të përpiqet të ruajë një `int` te vetë pointeri `p`, jo te `i`.

Gabim i rrezikshëm: thirrje pa & I

Situata

Nëse `decompose` pritet të marrë pointera, por i japim vlera, ndodh sjellje e rrezikshme.

Gabim i rrezikshëm: thirrje pa & II

Gabim

```
decompose(3.14159, i, d);
```

Çfarë ndodh? `p` është i painicializuar, prandaj nuk tregon në një adresë të vlefshme në memorie. Dereferencimi (`*p`) shkruan në një vend të panjohur dhe:

- programi mund të crash-ojë
- mund të korruptojë memorie (të prishë të dhëna diku tjetër)
- mund të sillet në mënyrë të paparashikueshme

Zgjidhja:

- bëjeni `p` të tregojë te një variabël ekzistuese (p.sh. `int x; p = &x;`)
- ose alokoni memorie (p.sh. me `malloc`) dhe kontrolloni që `p != NULL` para dereferencimit

Gabim i rrezikshëm: thirrje pa & III

Pse është problem

- Funksioni pret adresa
- Merr vlerat e `i` dhe `d`
- I interpreton ato si pointera
- Mund të shkruajë në zona të panjohura të memories

Praktikë e mirë

Përdor gjithmonë prototipe funksionesh: kompilatori zakonisht kap gabimin e tipit.

max_min: elementi më i madh

Qëllimi

Të gjejmë elementin më të madh dhe më të vogël në një array.

Prototipi

```
1 void max_min(int a[], int n, int *max, int *min);
```

Parametrat

- a[] = array i input-it
- n = numri i elementeve
- max, min = pointera ku ruhen rezultatet

max_min: kodi I

```c
#include <stdio.h>
#define N 10

void max_min(int a[], int n, int *max, int *min);

int main(void) {
    int b[N], i, big, small;

    printf("Shkruani %d numra: ", N);
    for (i = 0; i < N; i++)
        scanf("%d", &b[i]);

    max_min(b, N, &big, &small);

    printf("Me i madhi: %d\n", big);
    printf("Me i vogli: %d\n", small);

    return 0;
}
```

max_min: kodi II

```c
void max_min(int a[], int n, int *max, int *min) {
    int i;

    *max = *min = a[0];
    for (i = 1; i < n; i++) {
        if (a[i] > *max)
            *max = a[i];
        else if (a[i] < *min)
            *min = a[i];
    }
}
```

max_min: kodi III

Pika kyçe

*max dhe *min modifikojnë variablat big dhe small në main.

Ekzekutim

```
Shkruani 10 numra: 34 82 49 102 7 94 23 11 50 31
Me i madhi: 102
Me i vogli: 7
```

Rezultati

- big merr vlerën 102
- small merr vlerën 7

const për të mbrojtur argumentet

Kur përdoret

Ndonjëherë funksioni merr pointer vetëm për efikasitet, jo për të ndryshuar objektin.

Deklarimi

```
void f(const int *p);
```

Kuptimi

p është pointer drejt një **int konstant**: funksioni mund ta lexojë, por jo ta ndryshojë *p.

Gabim (kapet nga kompilatori)

```
*p = 0; /*** GABIM ***/
```

Format kryesore të `const` me pointera I

Ideja kryesore

Me pointera, `const` mund të mbrojë:

- **objektin** ku punton pointeri (`*p`)
- **vetë pointerin** (`p`)
- ose të dyja

Format kryesore të const me pointera II

Tre format kryesore

```
const int *p; /* ose: int const *p */
int *const p = &x;
const int *const p = &x;
```

Si lexohen

- `const int *p` = pointer drejt **int konstant**
- `int *const p` = **pointer konstant** drejt int
- `const int *const p` = pointer konstant drejt int konstant

const int *p

Deklarimi dhe përdorimi

```

int x = 10, y = 20;
const int *p = &x;

p = &y; /* lejohet */
```

Nuk lejohet

```
*p = 5; /* GABIM */
```

Kuptimi

Objekti (*p) është read-only përmes p, por pointeri mund të ndryshojë target.

int *const p

Deklarimi dhe përdorimi

```
int x = 10, y = 20;
int *const p = &x;

*p = 5; /* lejohet */
```

Nuk lejohet

```
p = &y; /* GABIM */
```

Kuptimi

Pointeri është fiks, por objekti mund të ndryshohet.

const int *const p

Deklarimi dhe përdorimi

```
int x = 10, y = 20;
const int *const p = &x;
```

Nuk lejohet

```
*p = 5; /* GABIM */
p = &y; /* GABIM */
```

Kuptimi

As pointeri, as objekti nuk ndryshohen përmes p.

Funksion që kthen pointer I

Ideja

Funksionet në C mund të kthejnë edhe pointera.

Shembull

```
int *max(int *a, int *b) {
    if (*a > *b)
        return a;
    else
        return b;
}
```

Funksion që kthen pointer II

Përdorimi

```
int *p, i, j;
/* ... */
p = max(&i, &j);
```

Rezultati

Pas thirrjes, `p` do të tregojë ose te `i` ose te `j`, sipas vlerës më të madhe.

Mos ktheni pointer drejt variabli lokal automatik

Gabim i zakonshëm

```c
int *f(void) {
    int i;
    /* ... */
    return &i; /* GABIM */
}
```

- `i` është variabël lokal automatik
- sapo funksioni përfundon, `i` nuk ekziston më
- pointeri i kthyer bëhet **i pavlefshëm** (dangling pointer)

Pointer drejt elementeve të array-it

Ideja

Pointerat mund të tregojnë edhe drejt elementeve të array-ve.

Shembull

```
int *find_middle(int a[], int n) {
    return &a[n / 2];
}
```

Interpretim

Funksioni kthen adresën e elementit në mes të array-it `a`.

Shprehje tipike

`&a[i]` është pointer drejt elementit `a[i]`.

Përmbajtja

1 Hyrje

2 Deklarimi i pointer-ave

3 Operatorët & dhe *

4 Caktimi i Pointerave

5 Pointer-at dhe Funksionet

6 Prioriteti i operatorëve me *

7 Përmbledhje

Prioriteti i operatorëve: pse ngatërrohemi?

Ideja

Në C, shprehjet me pointera si `*p++` ose `(*p)++` duken të ngjashme, por nuk bëjnë të njëjtën gjë.

Rregulli kryesor

- **Postfix** ++ (si te `p++`) ka prioritet më të lartë se `*`
- Prandaj:

$$*p{+}{+} \equiv *(p{+}{+})$$

- Nëse duam të rrisim **vlerën** ku punton pointeri:

$$(*p){+}{+}$$

Gabim tipik konceptual

Studentët shpesh lexojnë `*p++` si `(*p)++`, por kjo është **gabim**.

*p++

Shembull

```
1
2 int a[] = {10, 20, 30};
3 int *p = a;
4
5 printf("%d\n", *p++); /* printon 10 */
6 printf("%d\n", *p); /* tani printon 20 */
```

Çfarë ndodh?

- lexohet vlera aktuale (10)
- pastaj pointeri p kalon te elementi tjetër

(*p)++

Shembull

```c
int a[] = {10, 20, 30};
int *p = a;

(*p)++; /* rrit a[0] */
printf("%d\n", *p); /* 11 */
printf("%d\n", a[0]); /* 11 *
```

Çfarë ndodh?

- p nuk lëviz
- ndryshon **vlera** ku punton p

Shembuj të tjerë të zakonshëm: *++p dhe ++*p

Shembull

```c
int a[] = {10, 20, 30};
int *p = a;

printf("%d\n", *++p); /* 20 -> p leviz te a[1], pastaj dereferencohet */
printf("%d\n", ++*p); /* 21 -> rritet a[1], pastaj printohet */
printf("%d\n", a[1]); /* 21 */
```

Si t'i lexosh

- `*++p` ≡ `*(++p)` (ndryshon pointeri)
- `++*p` ≡ `++(*p)` (ndryshon vlera)

Kujdes

Këto forma janë të sakta, por për fillestarë shpesh janë të vështira për tu lexuar. Kur kodi bëhet i paqartë, përdor kllapa dhe/ose shkruaj dy rreshta.

Gabime tipike dhe praktikë e mirë I

Gabime tipike

- Të mendosh se `*p++` është njësoj si `(*p)++`
- Të harrosh që `p++` ndryshon pointerin (jo domosdoshmërisht vlerën)
- Të përdorësh forma të shkurtra pa qenë i sigurt për leximin

Gabime tipike dhe praktikë e mirë II

Praktikë e mirë (më e lexueshme)

```
/* ne vend te: x = *p++; */
x = *p;
p++;

/* ne vend te: (*p)++; */
*p = *p + 1;
```

Rregull praktik

Kur ke dyshim: **vendos kllapa** dhe lexo shprehjen hap pas hapi.

Përmbajtja

1 Hyrje

2 Deklarimi i pointer-ave

3 Operatorët & dhe *

4 Caktimi i Pointerave

5 Pointer-at dhe Funksionet

6 Prioriteti i operatorëve me *

7 Përmbledhje

Përmbledhje e leksionit

Çfarë mësuam

- Pointeri është një adresë në memorie
- `&` merr adresën, `*` dereferencon
- `*p` mund të jetë alias për një variabël
- Pointerat përdoren për të modifikuar argumentet e funksioneve
- Funksionet mund të kthejnë pointera

Rregulla kritike

- Mos dereferenconi pointera të painicializuar
- Mos ktheni pointer drejt variablave lokalë automatikë
- Kujdes me funksione si `scanf`: prisni pointer kur kërkohet pointer

Faleminderit!

Pyetje?